TUGAS PRAKTIKUM 2

Mata Kuliah: Pemrograman Basis Data

Materi: Struktur blok (Anonymous Block), Deklarasi Variabel, Konstanta, dan Tipe Data Komposit (Record/ %ROWTYPE)

Studi Kasus Validasi Data Akademik Mahasiswa Menggunakan Blok Prosedural

Disusun oleh : Lilis (IK2411012)
: Mifta Auliya (IK2411028)
Program Studi : Informatika
Universitas : Universitas Mega Buana Palopo
Dosen Pengampu : Abdul Malik, S.Kom., M.Cs.

Tahun 2026

2. Tujuan Pratikum

Praktikum ini bertujuan untuk memahami konsep dasar blok prosedural dalam pemrograman basis data, serta melatih kemampuan mahasiswa dalam mendeklarasikan variabel, menggunakan logika percabangan, dan menghasilkan output yang sistematis berdasarkan data akademik mahasiswa.

3. Deskripsi Singkat Studi Kasus

Studi kasus ini membahas tentang proses validasi data akademik mahasiswa sebelum pengisian KRS. Program dibuat untuk mengecek kelayakan mahasiswa berdasarkan data seperti SKS, IPK, dan status pembayaran. Selain itu, program juga dapat membandingkan dua mahasiswa untuk menentukan performa akademik yang lebih baik.

4. Dasar Teori (Singkat)

Blok prosedural adalah struktur program yang terdiri dari bagian deklarasi dan eksekusi, biasanya ditulis menggunakan BEGIN...END. Dalam MySQL, blok ini diimplementasikan melalui stored procedure.

Komponen penting:

- Variabel → menyimpan data sementara
- IF → logika percabangan
- CASE → alternatif kondisi
- CONCAT → menggabungkan teks

Blok prosedural membantu membuat program lebih terstruktur dan mudah dijalankan berulang.

5. KODE PROGRAM

1 Bagian A – Identitas Mahasiswa

1. Buat Tabel+Data

```
CREATE TABLE mahasiswa (
    nim VARCHAR(20) PRIMARY KEY,
    nama VARCHAR(50),
    semester INT,
    prodi VARCHAR(50)
);
INSERT INTO mahasiswa VALUES
('IK2411028','Mifta Auliya',4,'Informatika'),
('IK2411012','Lilis',4,'Informatika');
```

2. Storet Procedure Bagian A

```
DELIMITER //

DROP PROCEDURE IF EXISTS identitas_mhs_tabel //

CREATE PROCEDURE identitas_mhs_tabel()
BEGIN

    -- =========================
    -- Konstanta kampus
    -- =========================

    DECLARE kampus VARCHAR(100) DEFAULT 'Universitas Mega Buana
Palopo';

    -- =========================
    -- Menampilkan identitas
    -- =========================

    SELECT CONCAT(
        'Mahasiswa ', nama, ' (', nim, ') dari Program Studi ',
prodi,
        ' terdaftar di ', kampus, ' pada semester ', semester,
'.'
    ) AS identitas
    FROM mahasiswa;

END //

DELIMITER ;
```

3. Jalankan

```
CALL identitas_mhs_tabel();
```

4. Hasil Output

```
Mahasiswa Lilis (IK2411012) dari Program Studi Informatika terdaftar
di Universitas Mega Buana Palopo pada semester 4.

Mahasiswa Mifta Auliya (IK2411028) dari Program Studi Informatika terdaftar
di Universita
```

Program menggunakan tabel `mahasiswa` untuk menyimpan data, kemudian ditampilkan dalam bentuk kalimat menggunakan fungsi `CONCAT`. Variabel `kampus` digunakan sebagai konstanta untuk melengkapi informasi identitas mahasiswa.

2. Bagian B – Validasi Data Akademik

1. Tambahan Tabel Akademik

```
CREATE TABLE akademik (

    nim VARCHAR(20),

    sks INT,

    ipk DECIMAL(3,2),

    status_pembayaran VARCHAR(10)

);

INSERT INTO akademik VALUES

('IK2411028',16,3.20,'LUNAS'),

('IK2411012',20,3.60,'LUNAS');
```

2. Stored Procedure Bagian B

```
DELIMITER //

DROP PROCEDURE IF EXISTS bagianB_validasi //

CREATE PROCEDURE bagianB_validasi()
BEGIN

    -- =========================
    -- Validasi & Kategori
    -- =========================

    SELECT

        -- Status Data
        CONCAT('Status data: ',
            CASE
                WHEN a.status_pembayaran = 'LUNAS'
                     AND m.semester > 0
                     AND a.sks > 0
                THEN 'Valid'
                ELSE 'Tidak Valid'
            END
        ) AS status_data,

        -- Beban Studi
        CONCAT('Beban studi: ',
            CASE
                WHEN a.sks BETWEEN 1 AND 12 THEN 'Ringan'
```

```
                WHEN a.sks BETWEEN 13 AND 18 THEN 'Sedang'
                ELSE 'Padat'
            END
        ) AS beban_studi,

        -- Performa Akademik
        CONCAT('Performa akademik: ',
            CASE
                WHEN a.ipk >= 3.50 THEN 'Sangat Baik'
                WHEN a.ipk >= 3.00 THEN 'Baik'
                WHEN a.ipk >= 2.50 THEN 'Cukup'
                ELSE 'Perlu Pembinaan'
            END
        ) AS performa

    FROM mahasiswa m
    JOIN akademik a ON m.nim = a.nim
    WHERE m.nim IN ('IK2411028','IK2411012');

END //

DELIMITER ;
```

3. Cara Jalankan

```
CALL bagianB_validasi();
```

4. Hasil Output

```
Status data: Valid
Beban studi: Sedang
Performa akademik: Baik
```

kenapa pakai IF bukan CASE?”Jawab:karena IF digunakan dalam blok prosedural untuk pengolahan logika berbasis variabel

3. Bagian C – Ringkasan Kelayakan KRS

```
DELIMITER //

DROP PROCEDURE IF EXISTS bagianC_krs //

CREATE PROCEDURE bagianC_krs()

BEGIN

    -- ========================

    -- Ringkasan KRS

    -- ========================

    SELECT CONCAT(

        'Mahasiswa ', m.nama, ' dengan NIM ', m.nim,

        ' dinyatakan ',
```

```
-- Status KRS
CASE
    WHEN a.status_pembayaran = 'LUNAS'
         AND m.semester > 0
         AND a.sks > 0
    THEN 'layak mengambil KRS'
    ELSE 'tidak layak mengambil KRS'
END,
'. Beban studi berada pada kategori ',
-- Kategori SKS
CASE
    WHEN a.sks BETWEEN 1 AND 12 THEN 'Ringan'
    WHEN a.sks BETWEEN 13 AND 18 THEN 'Sedang'
    ELSE 'Padat'
END,
' dengan performa akademik ',
-- Kategori IPK
CASE
    WHEN a.ipk >= 3.50 THEN 'Sangat Baik'
    WHEN a.ipk >= 3.00 THEN 'Baik'
    WHEN a.ipk >= 2.50 THEN 'Cukup'
    ELSE 'Perlu Pembinaan'
END,
' (',
-- Alasan
CASE
    WHEN a.status_pembayaran = 'LUNAS'
```

```
                AND m.semester > 0

                AND a.sks > 0

            THEN 'karena pembayaran lunas dan SKS memenuhi syarat'

            ELSE 'karena terdapat data yang tidak memenuhi syarat'

        END,

        ').'

    ) AS hasil

    FROM mahasiswa m

    JOIN akademik a ON m.nim = a.nim

    WHERE m.nim IN ('IK2411028','IK2411012');

END //

DELIMITER ;
```

2. Jalankan

```
CALL bagianC_tabel();
```

3. Output

```
Mahasiswa Uminati dengan NIM IK2411011 dinyatakan layak mengambil KRS.
Beban studi berada pada kategori Sedang dengan performa akademik Baik
(karena pembayaran lunas
```

4. Bagian D – Analisis Dua Mahasiswa Buat satu program yang membandingkan

1. .Bagian D – Stored Procedure

```
DELIMITER //

DROP PROCEDURE IF EXISTS bagianD_perbandingan //

CREATE PROCEDURE bagianD_perbandingan()
BEGIN

    -- ========================
    -- 1. Tampilkan data mahasiswa
    -- ========================

    SELECT
        m.nama,
        m.nim,
        m.semester,
        a.sks,
        a.ipk,
        a.status_pembayaran
```

```
    FROM mahasiswa m
    JOIN akademik a ON m.nim = a.nim
    WHERE m.nim IN ('IK2411028','IK2411012');

    -- ========================
    -- 2. Kesimpulan perbandingan
    -- ========================

    SELECT
    CASE
        -- Bandingkan IPK
        WHEN a1.ipk > a2.ipk THEN
            CONCAT(m1.nama, ' memiliki performa akademik lebih
baik dibanding ', m2.nama)

        WHEN a2.ipk > a1.ipk THEN
            CONCAT(m2.nama, ' memiliki performa akademik lebih
baik dibanding ', m1.nama)

        -- Jika IPK sama → bandingkan SKS
        ELSE
            CASE
                WHEN a1.sks > a2.sks THEN
                    CONCAT(m1.nama, ' lebih baik karena jumlah
SKS lebih tinggi')
                WHEN a2.sks > a1.sks THEN
                    CONCAT(m2.nama, ' lebih baik karena jumlah
SKS lebih tinggi')
                ELSE
                    'Kedua mahasiswa memiliki performa akademik
yang sama'
            END
    END AS kesimpulan

    FROM mahasiswa m1
    JOIN akademik a1 ON m1.nim = a1.nim,
         mahasiswa m2
    JOIN akademik a2 ON m2.nim = a2.nim

    WHERE m1.nim = 'IK2411028'
      AND m2.nim = 'IK2411012';

END //

DELIMITER ;
```

2. Jalankan

```
CALL bagianD_perbandingan();
```

3. Contoh Output

- Data Mahasiswa:

```
Lilis | IK2411012 | 4 | 18 | 3.40 | LUNAS
Mifta Auliya | IK2411028 | 4 | 20 | 3.60 | LUNAS
```

- Kesimpulan:

```
Mifta Auliya memiliki performa akademik lebih baik dibanding Lilis
```

H. Ketentuan Tambahan

1. Skenario Uji

Skenario 1 – Data Valid

Data:

- Status pembayaran: LUNAS
- Semester: 4
- SKS: 18
- IPK: 3.40

Kode:

```
UPDATE mahasiswa
SET semester = 4
WHERE nim = 'IK2411012';

UPDATE akademik
SET sks = 18, ipk = 3.40, status_pembayaran = 'LUNAS'
WHERE nim = 'IK2411012';

CALL bagianC_tabel();
```

Output:

```
Mahasiswa Lilis dengan NIM IK2411012 dinyatakan layak mengambil KRS.
Beban studi berada pada kategori Sedang dengan performa akademik Baik
(karena pembayaran lunas dan SKS memenuhi syarat).
```

Skenario 2 – Tidak Valid (Pembayaran Belum Lunas)

Data:

- Status pembayaran: BELUM
- Semester: 4
- SKS: 18

Kode:

```
UPDATE akademik
SET status_pembayaran = 'BELUM'
WHERE nim = 'IK2411012';

CALL bagianC_tabel();
```

Output:

```
Mahasiswa Lilis dengan NIM IK2411012 dinyatakan tidak layak mengambil KRS.
Beban studi berada pada kategori Sedang dengan performa akademik Baik
(karena terdapat data yang tidak memenuhi syarat).
```

Skenario 3 – Tidak Valid (SKS = 0)

Data:

- Status pembayaran: LUNAS
- Semester: 4
- SKS: 0

Kode:

```
UPDATE akademik
SET sks = 0, status_pembayaran = 'LUNAS'
WHERE nim = 'IK2411012';

CALL bagianC_tabel();
```

Output:

```
Mahasiswa Lilis dengan NIM IK2411012 dinyatakan tidak layak mengambil KRS.
Beban studi berada pada kategori Padat dengan performa akademik Baik
(karena terdapat data yang tidak memenuhi syarat).
```

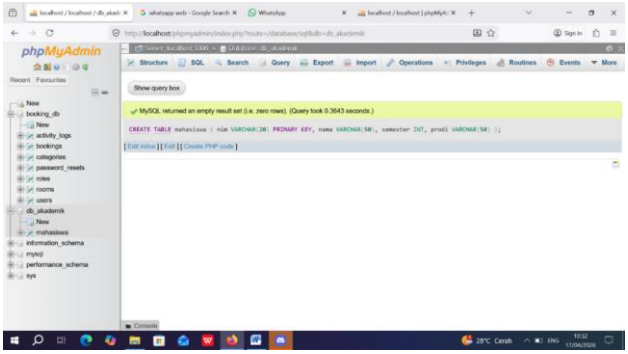
2. Analisis Hasil

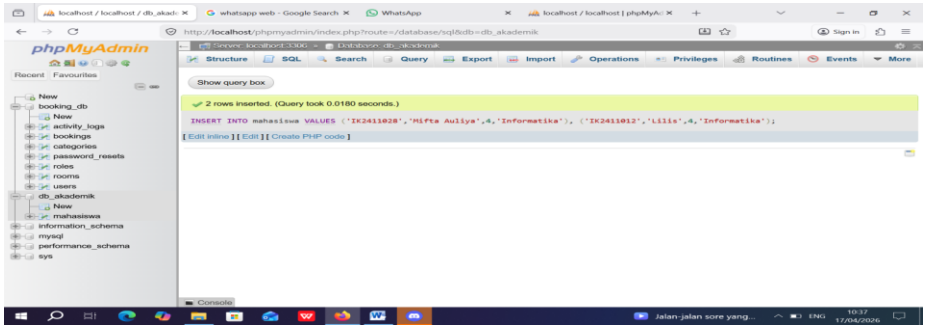
Berdasarkan hasil pengujian, terlihat bahwa program mampu memberikan keluaran yang berbeda sesuai dengan kondisi data yang dimasukkan.
Pada skenario pertama, mahasiswa dinyatakan layak mengambil KRS karena semua syarat terpenuhi, yaitu status pembayaran lunas, semester aktif, dan jumlah SKS lebih dari nol.
Pada skenario kedua, mahasiswa dinyatakan tidak layak karena status pembayaran belum lunas, meskipun SKS dan IPK memenuhi syarat.
Pada skenario ketiga, mahasiswa juga dinyatakan tidak layak karena jumlah SKS tidak memenuhi syarat, walaupun pembayaran sudah lunas.
Perbedaan hasil ini menunjukkan bahwa sistem validasi berjalan dengan baik dan mampu menyesuaikan keputusan berdasarkan kondisi data yang diberikan.

Program yang dibuat telah berhasil mengimplementasikan logika validasi akademik secara dinamis dan sesuai dengan aturan yang ditentukan.

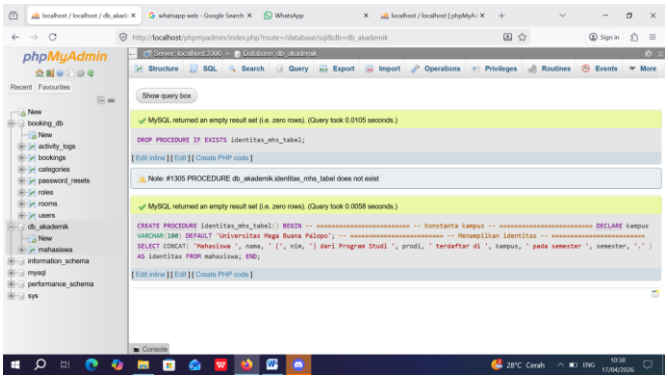
6. Screenshot Hasil Eksekusi

1. BUAT TABEL + DATA

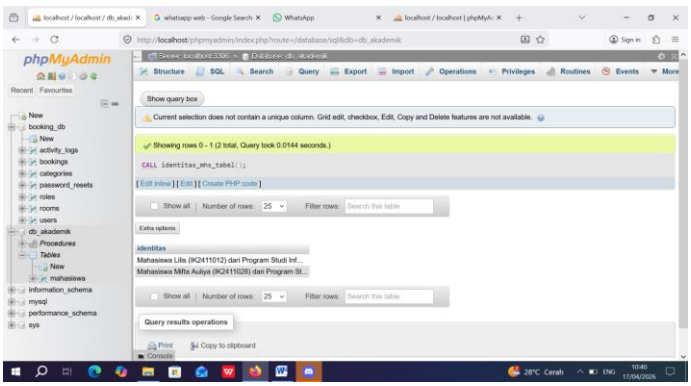




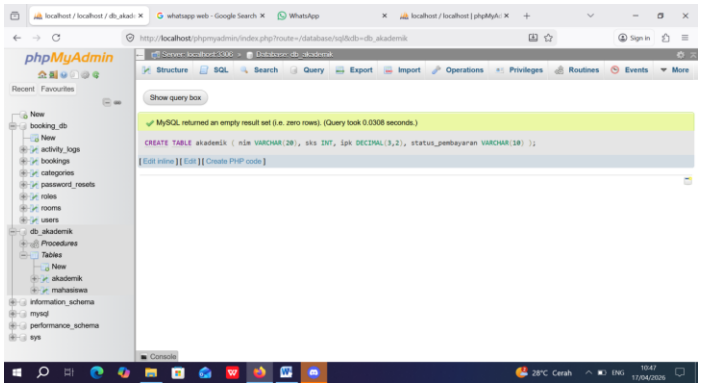
2. JALANKAN DAN HASIL OUTPUT



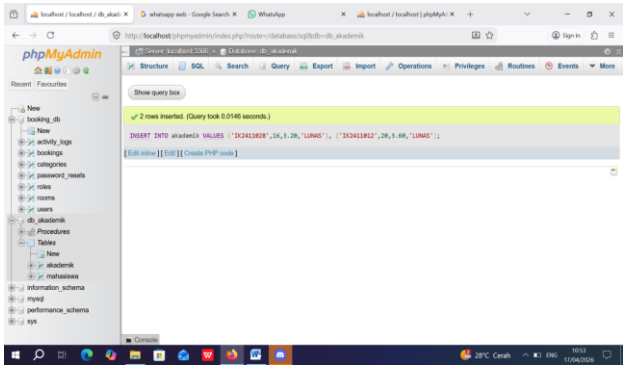
3. BAGIAN B – VALIDASI DATA AKADEMIK



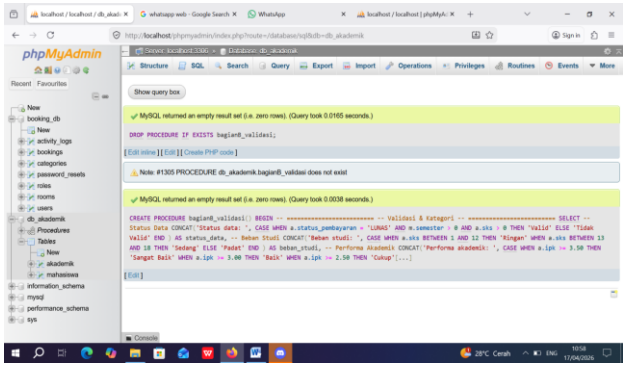
4. STORED PROCEDURE BAGIAN B (FINAL)



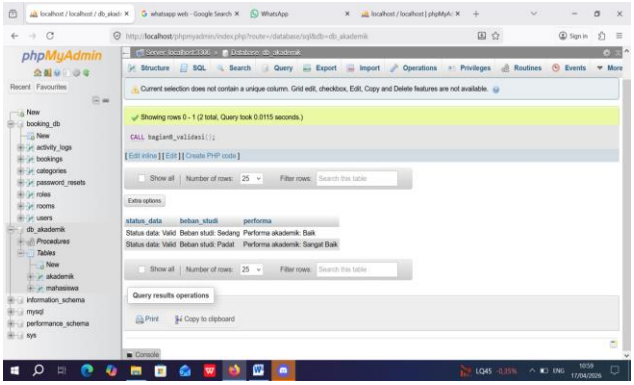
5. CARA MENJALANKAN DAN HASIL OUTPUT



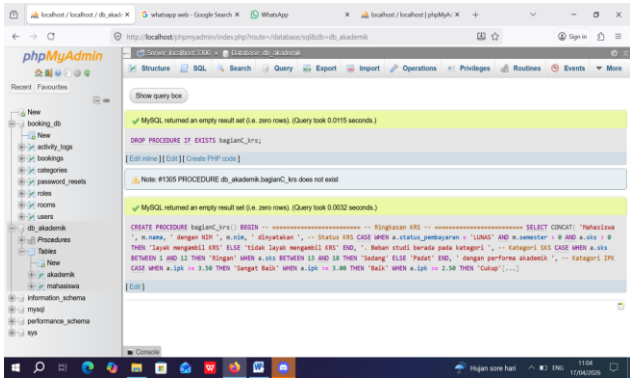
6. Bagian C – Ringkasan Kelayakan K



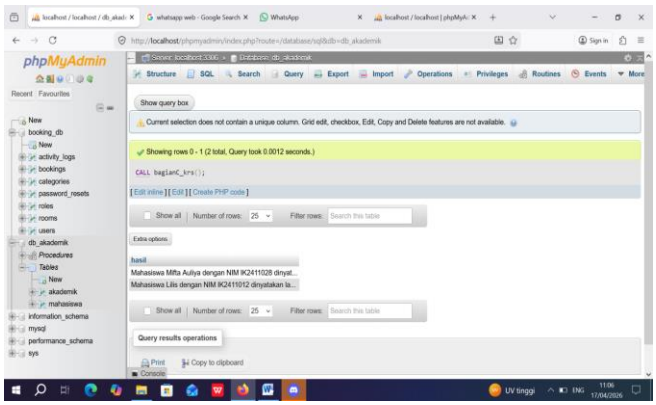
7. JALANKAN DAN HASIL OUTPUT



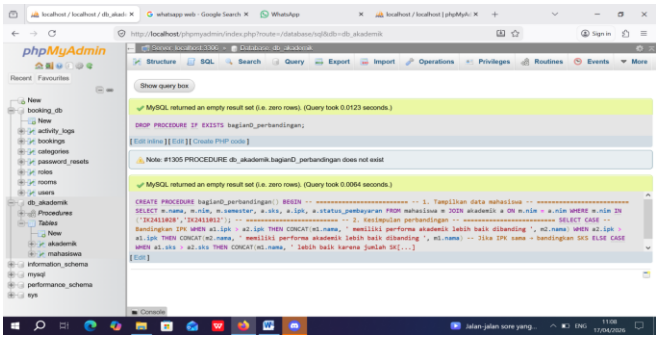
8. 8. ANALISIS DUA MAHASISWA BUAT SATU PROGRAM YANG MEMBANDINGKAN



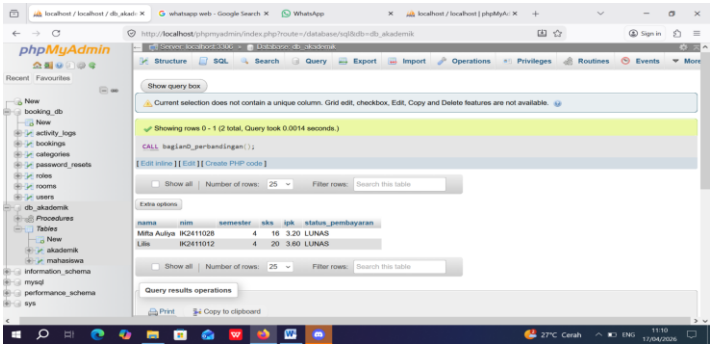
9. CARA MENJALANKAN DAN HASIL OUTPUT



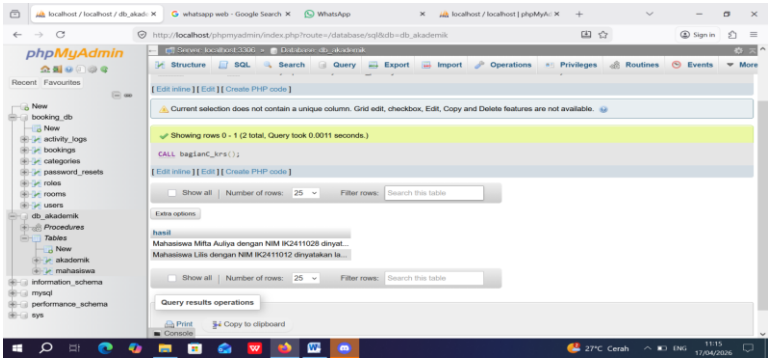
10. Skenario 1 – Data Valid

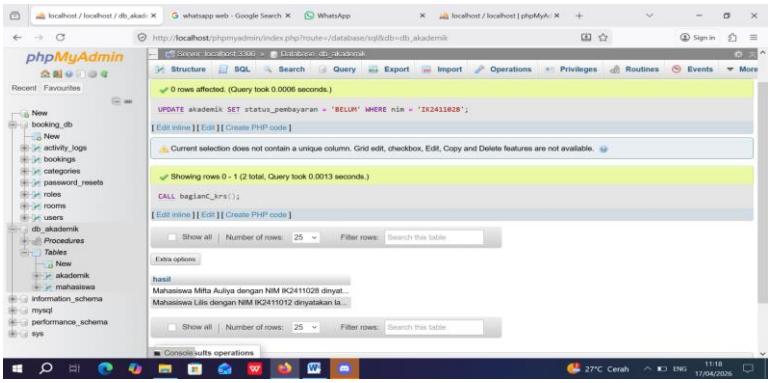


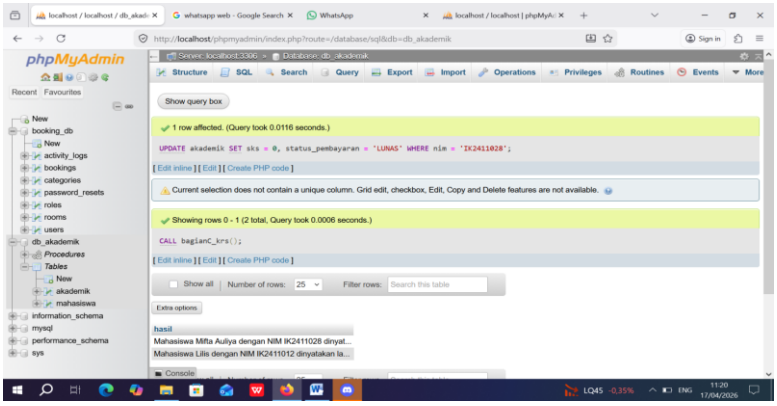
11. Skenario 2 – Tidak Valid (Pembayaran Belum Lunas)



12. Skenario 3 – Tidak Valid (SKS = 0)







13. Analisis Hasil

Berdasarkan hasil praktikum yang telah dilakukan pada Bagian A sampai dengan Bagian D, program yang dibuat mampu merepresentasikan konsep dasar blok prosedural dalam MySQL dengan baik. Pada Bagian A**,** program berhasil menampilkan identitas mahasiswa secara terstruktur menggunakan deklarasi variabel dan konstanta. Hal ini menunjukkan bahwa penggunaan variabel dalam blok program sudah sesuai dan dapat menghasilkan output yang rapi.

Pada Bagian B**,** program mampu melakukan validasi data akademik berdasarkan beberapa kondisi seperti status pembayaran, jumlah SKS, dan semester. Logika percabangan yang digunakan menghasilkan output yang berbeda sesuai dengan input yang diberikan. Selain itu, pengelompokan kategori beban studi dan performa akademik juga berjalan dengan baik berdasarkan ketentuan yang telah ditetapkan.

Pada Bagian C**,** program menggabungkan identitas mahasiswa dengan hasil validasi akademik sehingga menghasilkan informasi yang lebih lengkap mengenai kelayakan pengambilan KRS. Output yang dihasilkan menjadi lebih informatif karena tidak hanya menampilkan status validasi, tetapi juga memberikan kategori akademik dan alasan sederhana dari hasil tersebut.

Pada Bagian D, program berhasil membandingkan dua mahasiswa berdasarkan nilai IPK dan jumlah SKS. Logika perbandingan yang digunakan mampu menentukan mahasiswa dengan performa akademik yang lebih baik secara objektif. Hal ini menunjukkan bahwa program sudah mampu mengolah lebih dari satu data dan menghasilkan keputusan berdasarkan kondisi tertentu.

Selanjutnya, pada Bagian H (skenario uji)**,** terlihat adanya perbedaan hasil yang jelas antara data valid dan data tidak valid. Pada skenario valid, program menghasilkan keluaran bahwa mahasiswa

14. Kesimpulan

Dari praktikum yang telah dilakukan, dapat disimpulkan bahwa pemrograman basis data menggunakan blok prosedural sangat membantu dalam menyusun alur logika program secara sistematis dan terstruktur. Mahasiswa dapat memahami cara mendeklarasikan variabel, menggunakan konstanta, serta mengimplementasikan percabangan untuk menyelesaikan suatu studi kasus.

Melalui Bagian A hingga D, mahasiswa belajar bagaimana mengolah data sederhana menjadi informasi yang lebih bermakna, seperti validasi data akademik, pengelompokan kategori, hingga perbandingan antar mahasiswa. Selain itu, melalui pengujian pada beberapa skenario di Bagian H, mahasiswa dapat melihat secara langsung bagaimana perubahan input mempengaruhi output program.

Praktikum ini juga memberikan pemahaman dasar yang penting sebagai bekal untuk mempelajari materi lanjutan dalam pemrograman basis data, seperti stored procedure, function, dan trigger. Dengan demikian, tujuan pembelajaran dalam praktikum ini dapat dikatakan telah tercapai dengan baik.